

Multiple Courier Planning

Negin Odarbashi, negin.odarbashi@studio.unibo.it
Mohammad Pourtaheri, mohammad.pourtaheri@studio.unibo.it
Reihaneh Shahri, reihaneh.shahri@studio.unibo.it

February 2025

1 Introduction

In this project, we tackled the optimization problem using three different modeling approaches: MIP (Mixed-Integer Programming), CP (Constraint Programming) and SMT (Satisfiability Modulo Theories).

1.1 Group Members and Contributions

The work was divided among the group members as follows:

- **CP** was handled by Negin Odarbashi, who used constraint programming methods to model the problem and explored various search strategies.
- **SMT** was tackled by Mohammad Pourtaheri, who approached the problem using Satisfiability Modulo Theories to validate the feasibility of the solutions.
- **MIP** was implemented by Reihaneh Shahri, who focused on formulating the decision variables, constraints, and objective function related to the courier planning problem using MIP techniques.

The project began in September and was worked on intermittently over the course of several months. The main difficulties we encountered included integrating the different approaches, ensuring compatibility between the models, and fine-tuning the search strategies for optimal performance across all solvers. Despite these challenges, we were able to collaboratively refine our solutions and validate the results across the different methods.

1.2 Hardware Design

For the validation part, we ran all the models on a MacBook with an Apple M3 Pro chip and 18GB of RAM. The execution time is set to 300s. The search is used to guide the exploration of solutions and to further constrain variables.

2 CP Model

To solve the Multiple Courier Planning (MCP) problem using a Constraint Programming (CP) approach, we implemented a model in MiniZinc. This model defines courier routes while ensuring load feasibility, minimizing the maximum travel distance, and enforcing valid routing constraints.

2.1 Decision Variables

The model defines the following decision variables:

- **Integer Matrix** $H \in \mathbb{N}^{m \times (n+1)}$: Represents the courier routes, where:
 - $H[i, j] = k$: Courier i delivers item j and moves to location k .
 - $H[i, j] = n + 1$: Represents the depot, the starting and ending point for each courier.
- **Array of Natural Numbers** $couriers_loads \in [0, \max(l) + 1]$: Stores the total load assigned to each courier.
- **Array of Natural Numbers** $total_distance \in [0, \sum_{i,j \in \text{dis}} D[i, j]]$: Represents the total distance traveled by each courier c , calculated as:

$$total_distance[c] = \sum_{i \in \text{dis}, H[c, i] \neq i} D[i, H[c, i]].$$

- $H[c, i] \neq i$ ensures valid travel transitions.
- $D[i, H[c, i]]$ denotes the distance between consecutive locations.
- **Integer Variable** $obj \in [\text{low-bound}, \text{up-bound}]$:
 - Represents the maximum travel distance among all couriers, which we aim to minimize.

2.2 Objective Function

The model seeks to minimize the maximum distance traveled by any courier. To achieve this, the objective function is then formulated as:

$$obj = \max_{c \in \text{courier}} total_distance[c].$$

where:

- $total_distance[c]$ calculates the total distance traveled by each courier.
- The objective minimizes the maximum distance among all couriers, ensuring a balanced workload.

2.3 Constraints

The problem is subject to the following constraints:

- **Load Capacity Constraint:** Each courier's total assigned load must not exceed its capacity:

$$\forall i \in \text{courier}, \quad \sum_{j \in \text{items where } H[i, j] \neq j} s[j] \leq l[i].$$

- **Depot Visit Constraint:** Each courier must start and end at the depot:

$$\forall i \in \text{courier}, \quad H[i, n + 1] \neq n + 1.$$

- **Item Assignment Constraint:** Each item must be assigned to exactly one courier:

$$\forall j \in \text{items}, \quad \sum_{i \in \text{courier}} (H[i, j] \neq j) = 1.$$

- **Courier Subtour Constraint:** Each courier's route must form a valid subcycle:

$$\forall k \in \text{courier}, \quad \text{subcircuit}([H[k, i] \mid i \in \text{dis}]).$$

It ensures each node has one successor and one predecessor, enforces that each node belongs to one single cycle that covers all visited nodes.

- **Total Distance Calculation:** The total travel distance for each courier is computed as:

$$\forall c \in \text{courier}, \quad \text{total_distance}[c] = \sum_{i \in \text{dis where } H[c, i] \neq i} D[i, H[c, i]].$$

- **Lexicographical Ordering for Symmetry Breaking:** To reduce symmetry, courier routes are lexicographically ordered:

$$\forall k \in \text{courier} - 1, \quad \text{couriers_loads}[k] \geq \text{couriers_loads}[k + 1].$$

- **Objective Boundaries:** The maximum travel distance (*obj*) is constrained by:

$$\text{obj} = \max_{c \in \text{courier}} (\text{total_distance}[c]).$$

2.4 Validation

We evaluated the model using the Gecode and Chuffed solvers, comparing results with and without symmetry-breaking constraints.

2.5 Experimental Design

The search strategies used in MiniZinc include:

- **Restart Strategy (Luby Sequence):** This strategy follows the Luby sequence (200, 400, 200, etc.), helping escape local optima and improving search efficiency. It is used with all the solvers and models, in order to find optimal solution.
- **Variable Selection (Integer Search):**
We tested:
 - **First-Fail:** Focuses on variables with the smallest domain first; however, it was ineffective for this problem.
 - **Domain Weighted Degree (dom-w-deg):** Considers both domain size and constraint interactions, leading to better adaptability.
- **Value Selection (indomain-min):** Assigns the smallest possible value to the selected variable(current H[i, j]).
- **Relax-and-Reconstruct Heuristic:** The model applies the `relax_and_reconstruct` strategy with a relaxation factor of 70%, allowing partial solutions to be incrementally refined. This allows the solver to start by working with a simplified version of the problem (relaxing most of the constraints), find a quick solution, and then gradually refine that solution to meet the original, more complex set of constraints.

2.6 Experimental Results

ID	Gecode-sb	Chuffed-sb	Gecode-nosb	Chuffed-nosb	Gecode-int-nosb	Gecode-int-sb
1	14	14	14	14	14	14
2	226	226	226	226	226	226
3	12	12	12	12	12	12
4	220	220	220	220	220	220
5	206	206	206	206	206	206
6	322	322	322	322	322	322
7	167	172	167	167	167	167
8	186	186	186	186	186	186
9	436	436	436	436	436	436
10	244	244	244	244	244	244
11	902	N/A	898	N/A	425	353
12	636	N/A	636	N/A	346	346
13	748	1390	950	1456	448	430
14	1286	N/A	1257	N/A	594	555
15	1215	N/A	1133	N/A	536	570
16	370	1062	370	N/A	286	286
17	1366	N/A	1406	N/A	896	873
18	1082	N/A	1097	N/A	510	481
19	480	N/A	480	N/A	334	334
20	1466	N/A	1480	N/A	932	887
21	1120	N/A	1058	N/A	378	403

Table 1: Results using Gecode and Chuffed with(sb) and without symmetry(nosb) breaking using the integer search strategy(int) explained above.

- **Observations:**

- For smaller problems, the integer search and relax and reconstruct are less effective as they may introduce unnecessary overhead.
- For larger problems on the other hand, the integer search and relax and reconstruct enhance scalability and avoid development of suboptimal search spaces.

3 SMT Model

Our implementation of the SMT model leverages both CVC5 and Z3 for efficient solving and comparative analysis. The Z3 implementation uses add_soft constraints for diversity, while CVC5 relies on exclusion constraints. We modeled the problem using a binary matrix representation:

- The rows correspond to couriers.
- The columns represent products.

3.1 Decision Variables

The decision variables in our model correspond to the assignment of products to couriers. Each decision variable is binary, where:

- $M[i][j] = 1$ indicates that courier i is assigned to product j .

- $M[i][j] = 0$ indicates that courier i is not assigned to product j .

The solver will try to find a configuration of this matrix that satisfies all constraints and minimizes the total distance traveled by all couriers.

Theory used: The decision variables are modeled using Integer Programming (IP) with binary constraints, which is suitable for combinatorial optimization problems such as this one.

3.2 Heuristic Function

The objective function of the problem is to minimize the total distance traveled by each courier, while respecting the constraints on product assignments, capacities, and symmetry-breaking.

We do not directly formulate the objective function in the solver; instead, we calculate the total distance based on the assignments made by the solver and then minimize it using the search heuristic. For each solution candidate, the total distance is computed using the nearest-neighbor heuristic for the assigned products for each courier. This method is not optimal but computationally efficient (We tested both the exact optimal solution and the current heuristic, and found the latter to perform adequately within our computational constraints.).

3.3 Constraints

- **Binary Constraints:** The matrix entries $M[i][j]$ are binary variables, indicating whether a courier is assigned to a product. This ensures that each product is either assigned to a courier (1) or not (0). $M[i][j] \in \{0, 1\}$
- **Assignment Constraints:** Each product must be assigned to exactly one courier. This is enforced by the following constraint:

$$\sum_{i=1}^{\text{num_couriers}} M[i][j] = 1 \quad \text{for all products } j$$

This ensures that each product is delivered by one and only one courier.

- **Capacity Constraints:** Each courier has a maximum capacity, which must not be exceeded. The constraint is:

$$\sum_{j=1}^{\text{num_products}} M[i][j] \times \text{product_sizes}[j] \leq \text{capacities}[i]$$

This ensures that the total product sizes assigned to each courier do not exceed their capacity.

- **Symmetry-Breaking Constraints:** To reduce the search space and eliminate redundant solutions, we enforce a lexicographical ordering constraint on the first courier's product assignments. This ensures that solutions remain distinct, preventing the solver from exploring equivalent assignments. Empirically, we observed that this constraint allows our base model to find optimal solutions for instances 1–10 more efficiently.

The design of our symmetry-breaking constraints is heuristic, inspired by solver logs and patterns observed in well-known combinatorial optimization problems. Below, we detail the specific constraints used in different versions of our model.

- **Used in symmetry-breaking (SB) version:**

$$M[0][j] \leq M[0][j - 1]$$

This constraint ensures that if courier 0 is assigned product j , they must also be assigned all preceding products $j - 1, j - 2, \dots, 0$. This lexicographical ordering prevents equivalent solutions that differ only in index permutations. Initially, we tested this constraint in a version where couriers were sorted by capacity and items by weight, aiming to assign heavier loads to larger trucks. However, empirical results showed that enforcing this ordering constraint alone improved optimization performance, even without explicit sorting. By encouraging sequential assignment, this constraint helps structure product allocations, reducing redundant branching in the search tree.

- **Used in CVC5 version:**

$$M[i][j] \wedge M[i + 1][j + 1] \implies \neg M[i + 1][j]$$

In adapting the model for CVC5, we analyzed the pruning tree and observed frequent constraint violations, leading to excessive backtracking and increased solver runtime. This constraint refines the assignment structure by ensuring that if courier $i + 1$ takes product $j + 1$, they cannot simultaneously take product j . This adjustment further reduces symmetrical configurations that contribute to inefficient search space exploration.

Additionally, this formulation indirectly reinforces exclusivity in product assignments, ensuring a structured allocation without introducing redundant configurations.

3.4 Validation

- **Feasibility Check:** After solving the optimization problem, we ensure that all constraints are satisfied:
 - Each product is assigned to exactly one courier.
 - No courier exceeds their maximum capacity.
 - The symmetry-breaking constraints are respected.
- **Solution Verification:** The computed routes for each courier are validated by ensuring that they minimize the total distance based on the nearest-neighbor heuristic. This heuristic provides a reasonable approximation of the optimal solution.
- **Iterative Exploration:** The solver explores the solution space iteratively, adjusting the assignment matrix while checking for any improvements in the total distance. If the total distance of a new solution is better than the previous one, it is kept; otherwise, it is discarded.
- **Time and Iteration Limits:** To prevent excessive computation time, the solver is given a time limit and an iteration limit.

3.5 Experiments

- **Symmetry-Breaking (SB):** This is the main model we introduced with symmetry-breaking constraints.

- **Early Termination in Route Calculations (ET):** In this version, an early termination mechanism is added during route calculations. If the total distance exceeds the best-known solution (`max_distance`), the current route calculation is stopped early to avoid unnecessary computations. This approach is a heuristic method, which, if applied aggressively, can lead to non-optimal solutions, as we observed in instance 10. However, it led to finding more optimal solutions for instances 12, 13, 16, and 19. Also, this does not include symmetry-breaking.
- **CVC5:** In our CVC5 model, we applied basic constraints to address the problem domain, and it lacks symmetry-breaking constraints. Additionally, we included a constraint to ensure the exploration of diverse solutions. Notably, CVC5 demonstrated superior performance compared to the Z3 solvers, successfully finding solutions for all instances from 10 to 21 within significantly shorter timeframes.
- **LB-UB:** In this version, we apply lower-bound (LB) and upper-bound (UB) constraints to restrict the solution space:

$$LB = \max \left(\frac{\text{total_item_distances}}{\text{num_couriers}}, 0 \right)$$

The lower bound (LB) is defined based on the minimum workload distribution among couriers. Initially, the upper bound (UB) is set to infinity. Once the first feasible solution is found, UB is updated to the corresponding total distance, refining the search space dynamically.

During the exploration process, any plan in which a courier’s distance falls outside the range (LB,UB) is discarded. This pruning mechanism improves solver efficiency by eliminating unpromising solutions early.

Empirical results indicate that constraining the search within the LB-UB range often leads to improved solutions. Specifically, instances 7, 16, and 19 showed better optimization outcomes compared to other methods.

ID	SB	ET	LB-UP	CVC5
1	14	14	14	14
2	226	226	226	226
3	12	12	12	12
4	220	220	220	220
5	206	206	206	206
6	322	322	322	322
7	183	190	172	225
8	186	186	186	188
9	436	436	436	436
10	244	261	244	244
11	N/A	N/A	N/A	652
12	N/A	546	N/A	752
13	948	610	N/A	646
14	N/A	N/A	N/A	766
15	N/A	N/A	N/A	703
16	426	367	305	539
17	N/A	N/A	N/A	880
18	N/A	N/A	N/A	863
19	N/A	494	493	553
20	N/A	N/A	N/A	845
21	N/A	713	N/A	796

Table 2: Results obtained using Z3, ET, LB-UP, and CVC5 solvers, showcasing their performance with the corresponding instances as described above. Optimal solutions written in bold. All the 1-10 instances are optimal solutions.

4. MIP Model

This chapter presents a Mixed-Integer Programming (MIP) formulation for the Multiple Courier Planning (MCP) problem, implemented using AMPL. The model aims to optimize the allocation of delivery tasks among couriers while minimizing the total travel distance, respecting capacity constraints, and ensuring each item is delivered exactly once.

4.1 Decision Variables

- **Binary Route Decision Variable:**

$$X_{c,j,k} \in \{0, 1\}, \quad \forall c \in \text{couriers}, j, k \in \text{DS}$$

where $X_{c,j,k} = 1$ indicates that courier c travels directly from node j to node k .

- **Delivery Sequence Variable:**

$$H_k \in \{1, 2, \dots, n\}, \quad \forall k \in \text{items}$$

which represents the delivery sequence position of item k .

- **Objective Function Variable:**

$$\text{Obj} \in \mathbb{Z}$$

which represents the total distance traveled by any couriers.

4.2 Objective Function

The objective is to minimize the maximum travel distance among all couriers, ensuring a balanced workload distribution. The objective variable Obj represents the longest route taken by any courier.

$$\text{Obj} = \min \max_{c \in \text{couriers}} \sum_{j \in \text{DS}} \sum_{k \in \text{DS}} X[c, j, k] \cdot D[j, k]$$

where:

- $X[c, j, k]$ is a binary variable that equals 1 if courier c travels from location j to location k , and 0 otherwise.
- $D[j, k]$ represents the distance between locations j and k .

This objective is subject to the following constraint:

$$\sum_{x \in \text{DS}, y \in \text{DS}} X[c, x, y] \cdot D[x, y] \leq \text{Obj}, \quad \forall c \in \text{couriers}$$

This ensures that the total travel distance for each courier does not exceed Obj , enforcing a fairer distribution of workload among couriers. The objective variable is bounded as follows:

$$\text{low-bound} = \max_{i \in \text{items}} (D[n+1, i] + D[i, n+1])$$

$$\text{up-bound} = \text{max_distance} + \text{max_round_trip}$$

- The lower bound represents the shortest possible round-trip distance for the most distant item.
- The upper bound corresponds to the worst case where one courier handles all deliveries alone.

By minimizing Obj , we balance the workload across couriers, reducing inefficiencies and ensuring a fair distribution of travel distances.

4.3 Constraints

4.3.1 Core Constraints

- **Item Delivery:** Every item must be delivered exactly once:

$$\sum_{c \in \text{couriers}, j \in \text{DS}} X[c, j, k] = 1, \quad \forall k \in \text{items}.$$

- **Start and End at Depot:** Each courier starts and ends at the depot:

$$\sum_{y \in \text{DS}} X[c, n+1, y] = 1, \quad \forall c \in \text{couriers},$$

$$\sum_{x \in \text{DS}} X[c, x, n+1] = 1, \quad \forall c \in \text{couriers}.$$

- **Balanced Pick-up and Delivery:** If a courier picks up an item, it must also drop it off:

$$\sum_{y \in \text{DS}} X[c, x, y] = \sum_{y \in \text{DS}} X[c, y, x], \quad \forall c \in \text{couriers}, x \in \text{items}.$$

- **Courier Capacity:** The total size of delivered items must not exceed a courier's capacity:

$$\sum_{k \in \text{items}, j \in \text{DS}} X[c, j, k] \cdot \text{size}[k] \leq \text{capacity}[c], \quad \forall c \in \text{couriers}.$$

4.3.2 Delivery Order Constraints

- **Subtour elimination:** based on Miller-Tucker-Zemlin (MTZ) subtour elimination constraints, if courier c travels from item y to item x , then the visit order must satisfy:

$$H[x] \geq H[y] - M \cdot (1 - X[c, y, x]) + 1, \quad \forall c \in \text{couriers}, x, y \in \text{items}.$$

The constant M is used to linearize the nonlinear ordering constraint, making it suitable for mixed-integer programming (MIP) solvers. This technique, commonly employed in combinatorial optimization, enables flexibility in routing decisions by enforcing the visit sequence only when necessary. When a route does not include both items, M ensures the constraint is trivially satisfied. Typically set to a large value ($M = 1000$), it acts as a switch, conditioning the enforcement of the visit sequence based on the binary decision variable $X[c, y, x]$. This approach ensures an optimal solution without interfering with the optimization process, as discussed in the course lectures.

4.3.3 Objective Constraint

- **Objective Value Restriction:** The objective function must be at least the total distance traveled:

$$\sum_{x \in \text{DS}, y \in \text{DS}} X[c, x, y] \cdot D[x, y] \leq \text{Obj}, \quad \forall c \in \text{couriers}.$$

4.3.4 Lexicographical Ordering for Symmetry Breaking

- **Courier Ordering:** To prevent symmetrical solutions, we enforce an ordering condition:

$$\sum_{x, y \in \text{DS}} X[c, x, y] \geq \sum_{x, y \in \text{DS}} X[c + 1, x, y], \quad \forall c \in \{1, \dots, m - 1\}.$$

This ensures that courier routes are lexicographically ordered.

4.4 Model Validation

We implemented the MIP model in AMPL and tested it using the HiGHS and Gurobi solvers. The performance was evaluated with and without symmetry-breaking constraints to analyze their impact. The model was benchmarked on various instances, and solver performance was assessed based on optimality, runtime, and feasibility.

ID	highs	gurobi	gurobi-SB	highs-SB
1	14	14	14	14
2	226	226	226	226
3	12	12	12	12
4	220	220	220	220
5	206	206	206	206
6	322	322	322	322
7	167	167	167	167
8	186	186	186	186
9	436	436	436	436
10	244	244	244	244
11	N/A	N/A	N/A	N/A
12	N/A	611	N/A	N/A
13	N/A	440	N/A	N/A
14	N/A	N/A	N/A	N/A
15	N/A	N/A	N/A	N/A
16	286	286	286	N/A
17	N/A	N/A	N/A	N/A
18	N/A	N/A	N/A	N/A
19	N/A	364	N/A	N/A
20	N/A	N/A	N/A	N/A
21	N/A	N/A	N/A	N/A

Table 3: Results using HiGHS and Gurobi with and without symmetry breaking.

5 Conclusions

This project explored optimization techniques for the Multiple Courier Planning (MCP) problem using Constraint Programming (CP), Satisfiability Modulo Theories (SMT) and Mixed Integer Programming (MIP). The CP model, implemented with MiniZinc, effectively solved the problem using symmetry-breaking constraints and Hamiltonian sub-cycles. SMT models, leveraging solvers like CVC5 and Z3, provided scalability, with CVC5 excelling in efficiency for larger instances. MIP models, implemented in AMPL with HiGHS and Gurobi solvers, emphasized the use of symmetry-breaking and Big-M constraints. Experimental results showed that symmetry-breaking and heuristics improved solver efficiency, especially for larger instances. Early termination and tailored heuristics enabled efficient solution space exploration.

This work highlights the value of combining optimization methods to address complex logistics, laying the groundwork for future research on hybrid models and dynamic variables to improve scalability and adaptability.

References

- [1] AMPL Python API Documentation. *AMPL Optimization*.
Available at: <https://dev.ampl.com/ampl/python/index.html>
- [2] Z3 Solver Documentation. *Stanford University*.
Available at: <https://github.com/Z3Prover/z3/wiki/Documentation>
- [3] Finding Multiple Solutions in a Binary MIP.
Available at:
<https://orinanobworld.blogspot.com/2011/02/finding-multiple-solutions-in-binary.html>

- [4] MiniZinc Global Definitions. *MiniZinc*.
Available at: <https://docs.minizinc.dev/en/2.5.5/lib-globals.html>
- [5] Multi-vehicle Routing with Time Windows.
Available at: https://www.dcs.gla.ac.uk/~pat/cpM/minizincCPM/vrp/presentations/Tutorial10_Multi_vehicle_Routing_with_Time_Windows.pdf
- [6] Master's Thesis: On the Complexity of Optimizing Vehicle Routing Problems.
Available at: https://www.jacktex.eu/research/material/master_thesis.pdf
- [7] TSP MiniZinc Model from Minizinc examples. *Hakank's GitHub Repository*.
Available at: <https://github.com/hakank/hakank/blob/master/minizinc/tsp.mzn>